accumulator variable, ER. After looping through each frequency, line 3250 completes the calculation to make ER the mean squared error of the entire frequency response.

Lines 340 to 380 control the iteration loop of the program. Subroutine 2000 is where the changes to the recursion coefficients are made. The first action in this subroutine is to determine the current value of ER, and store it in another variable, EOLD (lines 2040 & 2050). After the subroutine updates the coefficients, the value of ER is again determined, and assigned to the variable, ENEW (lines 2270 and 2280).

The variable, MU, controls the iteration step size, just as in the previous neural network program. An advanced feature is used in this program: an *automated* adjustment to the value of MU. This is the reason for having the two variables, EOLD and ENEW. When the program starts, MU is set to the relatively high value of 0.2 (line 160). This allows the convergence to proceed rapidly, but will limit how close the filter can come to an optimal solution. As the iterations proceed, points will be reached where no progress is being made, identified by ENEW being *higher* than EOLD. Each time this occurs, line 370 reduces the value of MU.

Subroutine 2000 updates the recursion coefficients according to the steepest decent method: calculate the slope for each coefficient, and then change the coefficient an amount proportional to its slope. Lines 2080-2130 calculate the slopes for the "a" coefficients, storing them in the array, SA[ ]. Likewise, lines 2150-2200 calculate the slopes for the "b" coefficients, storing them in the array, SB[ ]. Lines 2220-2250 then modify each of the recursion coefficients by an amount proportional to these slopes. In this program, the proportionality constant is simply the step size, MU. No error term is required in the proportionality constant because there is only *one* example to be matched: the desired frequency response.

The last issue is how the program calculates the slopes of the recursion coefficients. In the neural network example, an *equation* for the slope was derived. This procedure cannot be used here because it would require taking the derivative *across* the DFT. Instead, a brute force method is applied: actually change the recursion coefficient by a small increment, and then directly calculate the new value of ER. The slope is then found as the change in ER divided by the amount of the increment. Specifically, the current value of ER is found in lines 2040-2050, and stored in the variable, EOLD. The loop in lines 2080-2130 runs through each of the "a" coefficients. The first action inside this loop is to add a small increment, DELTA, to the recursion coefficient being worked on (line 2090). Subroutine 3000 is invoked in line 2100 to find the value of ER with the modified coefficient. Line 2110 then calculates the slope of this coefficient as: $(ER - EOLD)/DELTA$. Line 2120 then restores the modified coefficient by subtracting the value of DELTA.

Figure 26-13 shows several examples of filters designed using this program. The dotted line is the desired frequency response, while the solid line is the